中国科学技术大学计算机学院

《数字电路实验》报告

实验题目：实验 08 信号处理及有限状态机

学生姓名：王琛

学生学号：PB20111696

完成日期：2021.12.2

计算机实验教学中心制

2020 年 09 月

【实验题目】

实验 08 信号处理及有限状态机

【实验练习】

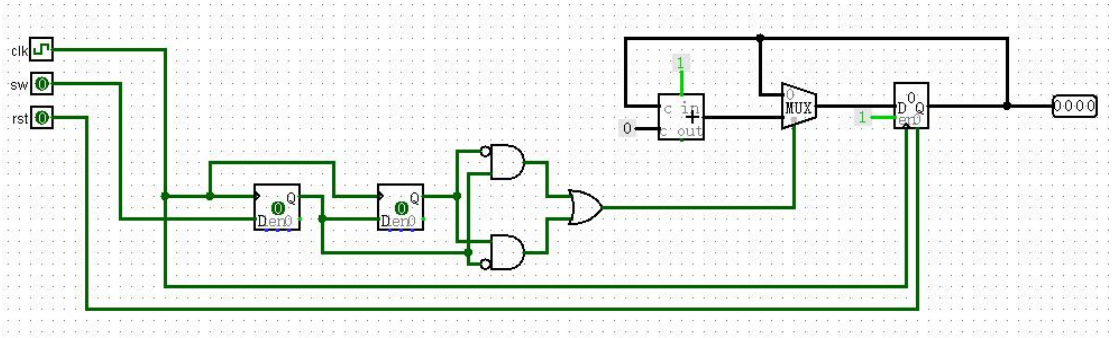
题目 1

将源代码中的时序逻辑部分拆分成两部分来分别生成次态信号以及将次态信号在时钟上升沿赋给现态信号，三段式代码如下

```
module practice1(
input clk,rst_n,
output led);
reg [1:0] curr_state;
reg [1:0] next_state;
always @(*) begin
    next_state = curr_state + 2'b1;
end
always@(posedge clk or posedge rst_n)
begin
    if(rst_n)curr_state <= 2'b0;
    else curr_state <= next_state;
end
assign led = (curr_state==2'b11) ? 1'b1 : 1'b0;
endmodule
```

题目 2

通过两个 D 触发器取 sw 的信号边沿，将得到的脉冲信号接入 MUX 来实现次态的选择，Logisim 中的电路设计如下



题目 3

使用实验步骤中的信号整形及去毛刺与取信号边沿技巧处理

按键信号，并使用生成的脉冲信号根据 sw[0]来对计数器进行改变，设计文件如下

```
module jitter_clr(
input clk,
input button,
output button_clean );
reg [3:0] cnt;
always@(posedge clk)
begin
    if(button==1'b0) cnt <= 4'h0;
    else if(cnt<4'h8) cnt <= cnt + 1'b1;
    else;
end
assign button_clean = cnt[3];
endmodule

module signal_edge(
input clk,
input button,
output button_edge);
reg button_r1,button_r2;
always@(posedge clk)
    button_r1 <= button;
always@(posedge clk)
    button_r2 <= button_r1;
assign button_edge = button_r1 & (~button_r2);
endmodule
```

```
module practice3(
    input clk,
    input [7:0] sw,
    input btn,
    output reg [2:0] hexplay_an,
    output reg [3:0] hexplay_data
);

reg [32:0] hexplay_cnt;
reg [7:0]counter;
wire btn_clean;
wire btn_edge;

jitter_clr jitter_clr(clk,btn,btn_clean);
signal_edge signal_edge(clk,btn_clean,btn_edge);

always@(posedge clk)
    if (hexplay_cnt >= (2000000 / 2)) hexplay_cnt <= 0;
    else hexplay_cnt <= hexplay_cnt + 1;

always@(posedge clk)
    if (hexplay_cnt == 0)
        if(hexplay_an >= 1) hexplay_an <= 0;
        else hexplay_an <= 1;
    else;
```

```
always@(posedge clk) begin
    if(sw[1])
        counter = 8'b0001_1111;
    else if(btn_edge)
        if(sw[0]) counter <= counter + 1;
        else counter <= counter - 1;
    else ;
end

always@(*) begin
    case(hexplay_an)
        0: hexplay_data = counter[3:0];
        1: hexplay_data = counter[7:4];
    endcase
end
```

约束文件如下

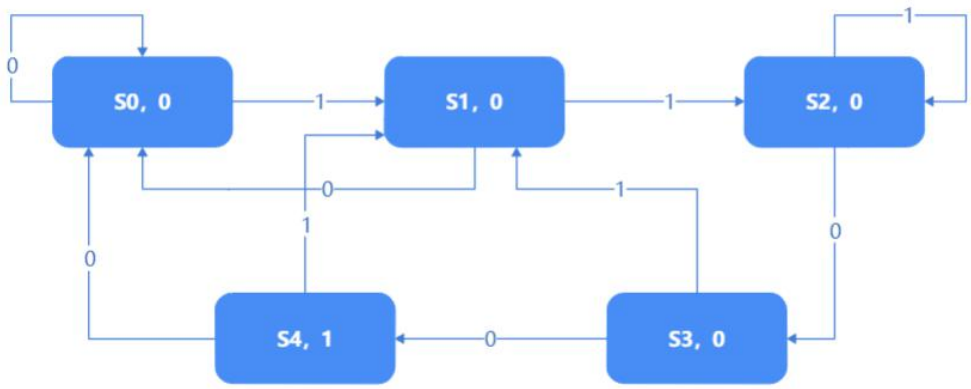
```
# Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }];

# FPGAOL SWITCH

set_property -dict { PACKAGE_PIN D14   IOSTANDARD LVCMOS33 } [get_ports { sw[0] }];
set_property -dict { PACKAGE_PIN F16   IOSTANDARD LVCMOS33 } [get_ports { sw[1] }];
set_property -dict { PACKAGE_PIN G16   IOSTANDARD LVCMOS33 } [get_ports { sw[2] }];
set_property -dict { PACKAGE_PIN H14   IOSTANDARD LVCMOS33 } [get_ports { sw[3] }];
set_property -dict { PACKAGE_PIN E16   IOSTANDARD LVCMOS33 } [get_ports { sw[4] }];
set_property -dict { PACKAGE_PIN F13   IOSTANDARD LVCMOS33 } [get_ports { sw[5] }];
set_property -dict { PACKAGE_PIN G13   IOSTANDARD LVCMOS33 } [get_ports { sw[6] }];
set_property -dict { PACKAGE_PIN H16   IOSTANDARD LVCMOS33 } [get_ports { sw[7] }];

# FPGAOL HEXPLAY

set_property -dict { PACKAGE_PIN A14   IOSTANDARD LVCMOS33 } [get_ports { hexplay_data[0] }];
set_property -dict { PACKAGE_PIN A13   IOSTANDARD LVCMOS33 } [get_ports { hexplay_data[1] }];
set_property -dict { PACKAGE_PIN A16   IOSTANDARD LVCMOS33 } [get_ports { hexplay_data[2] }];
set_property -dict { PACKAGE_PIN A15   IOSTANDARD LVCMOS33 } [get_ports { hexplay_data[3] }];
set_property -dict { PACKAGE_PIN B17   IOSTANDARD LVCMOS33 } [get_ports { hexplay_an[0] }];
set_property -dict { PACKAGE_PIN B16   IOSTANDARD LVCMOS33 } [get_ports { hexplay_an[1] }];
set_property -dict { PACKAGE_PIN A18   IOSTANDARD LVCMOS33 } [get_ports { hexplay_an[2] }];

# FPGAOL BUTTON & SOFT_CLOCK

set_property -dict { PACKAGE_PIN B18   IOSTANDARD LVCMOS33 } [get_ports { btn }];
```

题目 4

使用三段式实现序列检测的有限状态机，设计文件如下

```
module machine(
input clk,se,
output [2:0]condition,
output result);

reg [2:0] curr_state;
reg [2:0] next_state;
parameter S_0 = 3'b000;
parameter S_1 = 3'b001;
parameter S_2 = 3'b010;
parameter S_3 = 3'b011;
parameter S_4 = 3'b100;

initial begin
    curr_state = S_0;
end
always @(*) begin
    case (curr_state)
        S_0:  next_state = (se == 0) ? S_0 : S_1 ;
        S_1:  next_state = (se == 0) ? S_0 : S_2 ;
        S_2:  next_state = (se == 0) ? S_3 : S_2 ;
        S_3:  next_state = (se == 0) ? S_4 : S_1 ;
        S_4:  next_state = (se == 0) ? S_0 : S_1 ;
        default: next_state = S_0;
    endcase
end

always@(posedge clk)
begin
    curr_state <= next_state;
end

assign result = (curr_state == S_4) ? 1'b1 : 1'b0;
assign condition = curr_state;
endmodule
```

状态跳转图如下



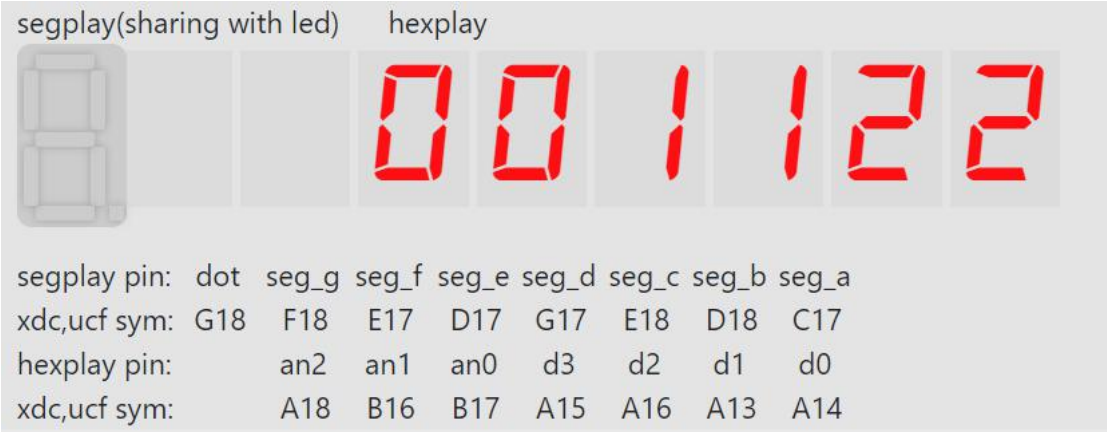
例化有限状态机并使用数码管显示最近输入数值，目标序列个

数，状态机编码，设计文件如下

```
module practice4(
    input clk,
    input [7:0] sw,
    input btn,
    output reg [2:0] hexplay_an,
    output reg [3:0] hexplay_data
);

reg [32:0] hexplay_cnt;
wire btn_clean,btn_edge,result;
wire [2:0] condition;
reg [3:0] counter;
reg [3:0] data;
wire result_edge;

jitter_clr jitter_clr(clk,btn,btn_clean);
signal_edge signal_edge1(clk,btn,btn_edge);
signal_edge signal_edge2(clk,result,result_edge);
machine machine(btn_edge,sw[0],condition,result);

initial begin
    counter = 0;
    data = 0;
end

always@(posedge clk)
    if (hexplay_cnt >= (2000000 / 6)) hexplay_cnt <= 0;
    else hexplay_cnt <= hexplay_cnt + 1;
```

```
always@(posedge clk)
    if (hexplay_cnt == 0)
        if(hexplay_an >= 5) hexplay_an <= 0;
        else hexplay_an <= hexplay_an + 1;
    else;

always@(posedge clk) begin
    if(result_edge)
        counter <= counter + 1;
    else ;
end

always @(posedge clk) begin
    if(btn_edge)
        data <= {data[2:0],sw[0]};
    else;
end

always@(*) begin
    case(hexplay_an)
        0: hexplay_data = counter;
        1: hexplay_data = condition;
        2: hexplay_data = data[0];
        3: hexplay_data = data[1];
        4: hexplay_data = data[2];
        5: hexplay_data = data[3];
    endcase
end

endmodule
```

在 FPGAOL 平台上进行测试，输入 “0011001110011”时，结果如下，其中从左到右前四位为最近输入数值 0011，第五位为状态编码 2（对应状态图中 S2），第六位为目标序列个数 2。



【总结与思考】

1. 请总结本次实验的收获

(1)掌握几种常见的信号处理技巧

(2)掌握有限状态机的设计方法

(3)学习使用有限状态机设计功能电路

2. 请评价本次实验的难易程度

本次实验题目量适中

3. 请评价本次实验的任务量

实验练习能够在规定时间内完成

4. 请为本次实验提供改进建议

修改 step5 代码中 rst 和 rst_n 不统一的问题